Atividade Prática 01 — API de Consulta de Capitais

Disciplina: Desenvolvimento Back-end
Tecnologias: Node.js, TypeScript e Express
Nível: Iniciante
Conteúdos de referência: Tutoriais 02, 03 e 04

---

1. Contextualização

Você foi contratado para desenvolver uma pequena API que permita consultar a capital de um estado brasileiro.

A aplicação deverá receber a sigla de um estado (UF) por meio de uma requisição HTTP e retornar, em formato JSON, o nome da capital correspondente.

Por exemplo, ao consultar a sigla SP, a API deverá retornar São Paulo.

O objetivo desta atividade é aplicar os conhecimentos adquiridos nos três primeiros tutoriais, construindo uma aplicação back-end funcional.

2. Objetivo

Desenvolver uma API utilizando Node.js, TypeScript e Express, aplicando os conceitos de:

- Inicialização de um projeto Node.js;
- Configuração do TypeScript;
- Criação e execução de um servidor Express;
- Definição de rotas HTTP;
- Utilização do método GET;
- Recebimento de parâmetros pela URL;
- Retorno de respostas no formato JSON;
- Utilização de códigos de status HTTP.

3. Requisitos funcionais

A API deverá atender aos seguintes requisitos:

RF01 — Consultar capital

A aplicação deverá disponibilizar uma rota que receba a sigla de um estado brasileiro e retorne sua capital.

RF02 — Reconhecer as siglas

A aplicação deverá reconhecer as siglas dos 26 estados brasileiros e do Distrito Federal.

RF03 — Ignorar diferenças entre maiúsculas e minúsculas

A consulta deverá funcionar independentemente de a sigla ser informada em letras maiúsculas ou minúsculas.

Exemplos: SP, sp ou Sp.

RF04 — Estado não encontrado

Caso seja informada uma sigla inexistente, a API deverá retornar uma resposta de erro com o código HTTP 404.

4. Requisitos técnicos

A aplicação deverá obrigatoriamente:

1. Ser desenvolvida em TypeScript.
2. Utilizar Node.js e Express.
3. Possuir um servidor HTTP funcional.
4. Utilizar o método GET para consulta.
5. Receber a sigla por meio de um parâmetro de rota.
6. Retornar os resultados em JSON.
7. Utilizar códigos de status HTTP adequados.
8. Manter os dados das unidades federativas em uma estrutura de dados em memória.

5. Especificação da API

Endpoint

GET /estados/:sigla

O parâmetro :sigla representa a unidade federativa que será consultada.

Exemplos de utilização

Exemplo 1 — Consulta válida

Requisição:

GET http://localhost:3000/estados/SP

Resposta esperada:

Código HTTP: 200 OK

```
{
  "sigla": "SP",
  "capital": "São Paulo"
}
```

Exemplo 2 — Outra consulta válida

Requisição:

GET http://localhost:3000/estados/MG

Resposta esperada:

Código HTTP: 200 OK

{
  "sigla": "MG",
  "capital": "Belo Horizonte"
}

Exemplo 3 — Sigla inexistente

Requisição:

GET http://localhost:3000/estados/XX

Resposta esperada:

Código HTTP: 404 Not Found

{
  "erro": "Estado não encontrado."
}

Exemplo 4 — Sigla em letras minúsculas

Requisição:

GET http://localhost:3000/estados/rj

Resposta esperada:

Código HTTP: 200 OK

{
  "sigla": "RJ",
  "capital": "Rio de Janeiro"
}

6. Etapas de desenvolvimento

Etapa 1 — Preparação do projeto

- Criar uma pasta para a atividade.
- Inicializar o projeto utilizando npm.
- Instalar e configurar as dependências necessárias.
- Configurar o TypeScript.

- Abrir o projeto no VS Code.

Etapa 2 — Criação do servidor

- Criar o arquivo de entrada da aplicação (entry point).
- Configurar o Express.
- Definir a porta do servidor.
- Iniciar o servidor HTTP.
- Verificar se a aplicação está funcionando.

Etapa 3 — Desenvolvimento da rota

- Criar a rota GET /estados/:sigla.
- Receber a sigla utilizando os parâmetros da requisição.
- Preparar uma estrutura de dados contendo as siglas e capitais.
- Implementar a consulta.
- Retornar a capital correspondente em JSON.
- Tratar siglas inexistentes.

Etapa 4 — Testes

Realizar testes utilizando Postman, Thunder Client ou outro cliente HTTP.

Testar obrigatoriamente:

Teste	Entrada	Resultado esperado
1	SP	São Paulo
2	RJ	Rio de Janeiro
3	MG	Belo Horizonte
4	BA	Salvador
5	AM	Manaus
6	df	Brasília
7	XX	Erro 404

7. Entrega

O aluno deverá entregar o link para o repositório da aplicação contendo:

1. O código-fonte completo da aplicação.
2. O arquivo package.json.
3. O arquivo tsconfig.json.

4. Uma breve descrição de como executar o projeto no arquivo readme.md
5. Evidências dos testes realizados, contendo as requisições e respostas da API. É possível usar print de telas com os testes executados. Os prints de tela devem estar dentro da pasta docs\testes
6. A API deverá ser criada em Python, devolvendo os Estados Americano (EUA) que começam com A e os Estados Canadenses e de outros países que começam com B.

8. Critérios de avaliação

Critério	**Pontuação**
Configuração correta do projeto Node.js e TypeScript	10,00
Inicialização e funcionamento do servidor Express	10,00
Implementação correta da rota GET	20,00
Recebimento e tratamento do parâmetro de rota	10,00
Consulta correta das capitais	20,00
Tratamento de siglas inexistentes e status HTTP	10,00
Testes e evidências de funcionamento	10,00
Organização e entrega do projeto	10,00
Total	**100,0**